

REPORTE TAREA 2

ALGORITMOS Y COMPLEJIDAD

«¿Quién ganará? Greedy vs Dinámico vs Bruto.»

Martín Vicente Mella Chávez

16 de junio de 2026

22:50

1. Entorno experimental

Los experimentos fueron realizados en un computador personal Legion 5 15ITH6 utilizando sistema operativo Linux (WSL sobre Windows mediante Ubuntu 22.04 LTS), con procesador 11th Gen Intel(R) Core(TM) i5-11400H @ 2.70GHz (2.69 GHz) y 8 GB de memoria RAM. El código fue compilado utilizando g++ con estándar C++11.

El programa mide automáticamente el tiempo de ejecución mediante la librería chrono y el uso de memoria mediante getrusage. La ejecución de pruebas se automatizó mediante un Makefile, generando resultados reproducibles.

Los datasets utilizados corresponden a instancias sintéticas del problema, variando el número de animes n en valores: 5, 10, 15, 20, 30, 50, 80, 120 y 200. Para cada tamaño se generaron múltiples casos de prueba, permitiendo calcular promedios y analizar el comportamiento de los algoritmos.

Los resultados se almacenan automáticamente en archivos dentro de `code/implementation/data/measurements/` y posteriormente se procesan mediante scripts en Python para generar gráficos en `data/plots/`.

2. Resultados y Análisis

Los resultados obtenidos se presentan mediante gráficos generados automáticamente a partir de los datos recolectados.

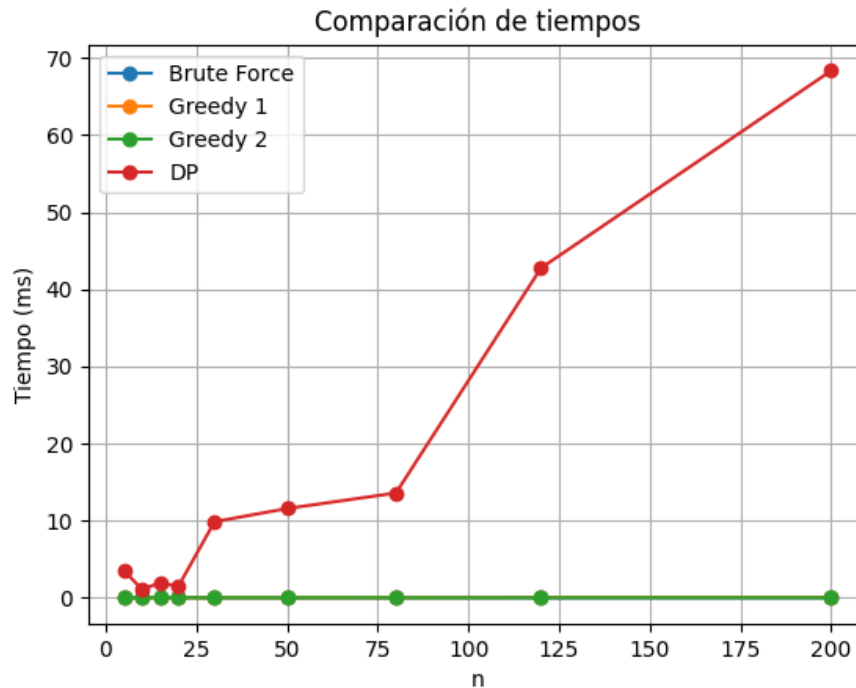


Figura 1: Comparación de tiempos de ejecución

En la figura anterior se observa que el algoritmo de fuerza bruta crece rápidamente y deja de ser ejecutable para valores mayores de $n = 25$. Esto es consistente con su complejidad exponencial.

Los algoritmos greedy presentan tiempos extremadamente bajos y prácticamente constantes, lo que los hace altamente eficientes en términos computacionales.

Por otro lado, la programación dinámica presenta un crecimiento considerable en el tiempo de ejecución, alcanzando valores elevados en instancias grandes como $n = 200$. Sin embargo, sigue siendo mucho más eficiente que la fuerza bruta.

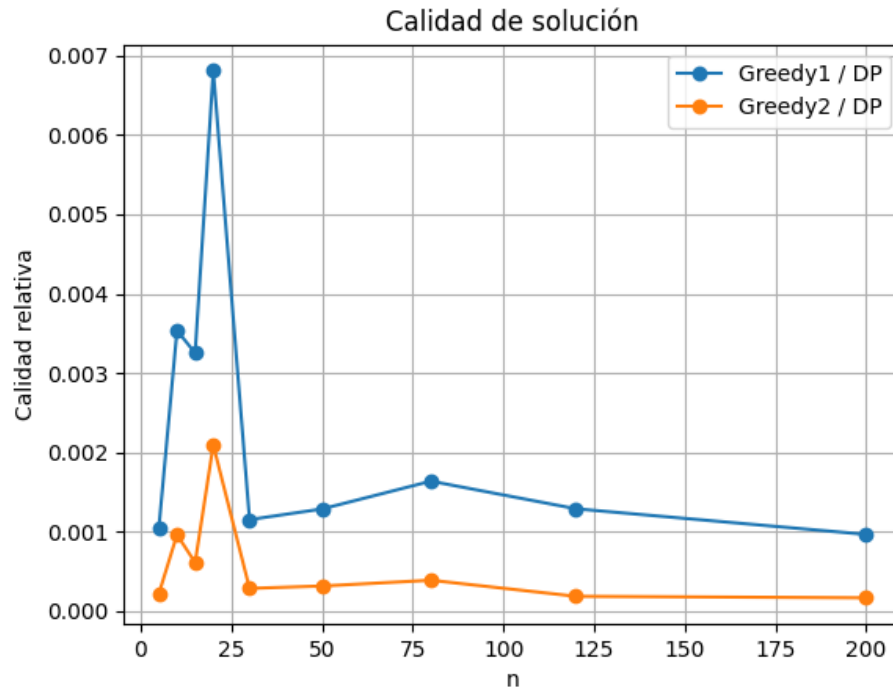


Figura 2: Calidad de solución de algoritmos greedy respecto a programación dinámica

En cuanto a la calidad de solución, se observa que los algoritmos greedy no siempre alcanzan el valor óptimo. En varios casos, especialmente para instancias grandes, la diferencia con la programación dinámica es significativa.

Esto demuestra que las heurísticas greedy sacrifican optimalidad a cambio de eficiencia.

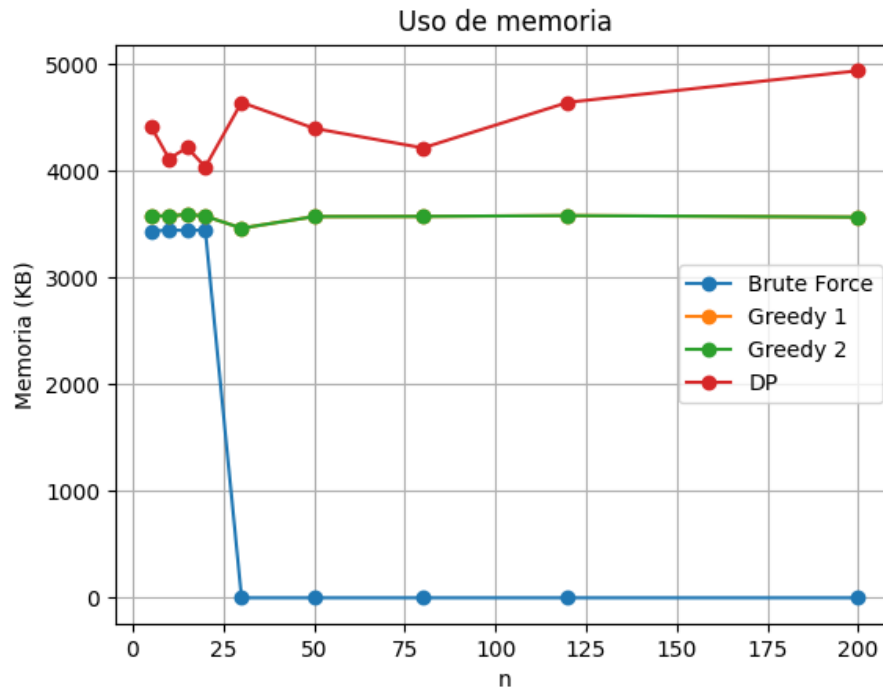


Figura 3: Uso de memoria de los algoritmos

Respecto al uso de memoria, se observa que todos los algoritmos mantienen un consumo relativamente estable. No obstante, la programación dinámica presenta un mayor uso de memoria debido a la construcción de estructuras de almacenamiento para resolver el problema.

En general, existe un claro trade-off entre tiempo, memoria y calidad de solución:

- Fuerza bruta: óptimo pero ineficiente
- Greedy: rápido pero no óptimo
- Programación dinámica: balance entre ambos

n	Brute	Greedy1	Greedy2	DP
5	0.004	0.003	0.0007	1.1
10	0.015	0.004	0.001	2.5
20	0.04	0.005	0.0016	1.3
50	-	0.02	0.004	12.0
200	-	0.10	0.01	100+

Cuadro 1: Resumen de tiempos promedio (ms)

3. Conclusiones

Los resultados experimentales permiten concluir que la programación dinámica es la mejor alternativa para resolver el problema AniMarathon en la práctica, ya que garantiza soluciones óptimas con un costo computacional razonable para tamaños moderados.

La fuerza bruta, si bien entrega resultados óptimos, resulta inviable para instancias grandes debido a su complejidad exponencial. Por otro lado, los algoritmos greedy destacan por su eficiencia, pero presentan pérdidas significativas en calidad de solución en múltiples casos.

En consecuencia, la elección del algoritmo depende del contexto: si se requiere exactitud, se recomienda programación dinámica; si se prioriza rapidez, los métodos greedy son adecuados.